

Measuring Algorithms

Numerical Methods

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory

Department of Electrical and Computer Engineering

Cullen College of Engineering

University of Houston

UNIVERSITY of **HOUSTON** | ECE

Approximations for Error

- Approximate e^x as a Maclaurin series:

$$e^x = \underbrace{1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \frac{x^4}{4!}}_{M(x)} + \epsilon$$

approximation
error

$$\epsilon = O(x^5)$$

- "Big-O" notation specifies how the term behaves as x changes
 - in this case ϵ **behaves like** x^5 when $x \rightarrow 0$ or $x \rightarrow \infty$
 - the final term is bounded by $O(x^5) \leq C|x|^5$ for some C
 - $C_1|x|^n < C_2|x|^{n+1}$ for large x **for any** C_1 and C_2
 - if $C_1 \gg C_2$, there is still an x for which $C_1|x|^n < C_2|x|^{n+1}$



Estimating Relative Errors

- Consider an algorithm for the exponential function based on its Maclaurin series:

```
1 float exp(float x, int N) {  
2     float ex = 0.0;  
3     float factorial = 1;  
4     for(int k = 0; k <= N; i++){  
5         factorial *= i;  
6         ex += pow(x, k) / factorial;  
7     }  
8     return ex;  
9 }
```

$$e^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots + O(x^N)$$

$$e^x = \sum_{k=0}^N \frac{x^k}{k!} + O(x^N)$$

$$\begin{aligned} \epsilon &= O(x^5) & \epsilon &= O\left(\frac{1}{2}x^5\right) \\ \epsilon &\leq C|x|^5 & \epsilon &\leq C\left|\frac{1}{2}x\right|^5 \\ & & \epsilon &\leq C\frac{1}{32}|x|^5 \end{aligned}$$

- How does the error differ when N and x change?

`exp(x, 5)` VS `exp(0.5*x, 5)`

`exp(x, 5)` VS `exp(2*x, 3)`

- We can adjust error bounds when comparing algorithms

Estimating Algorithmic Speed

- The same concept applies to the time to solve a numerical problem
- How many operations are required for $c = a + b$ where $a, b \in \mathbb{R}$?

1 addition, or 1 floating point operation (FLOP)

- What if $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$?

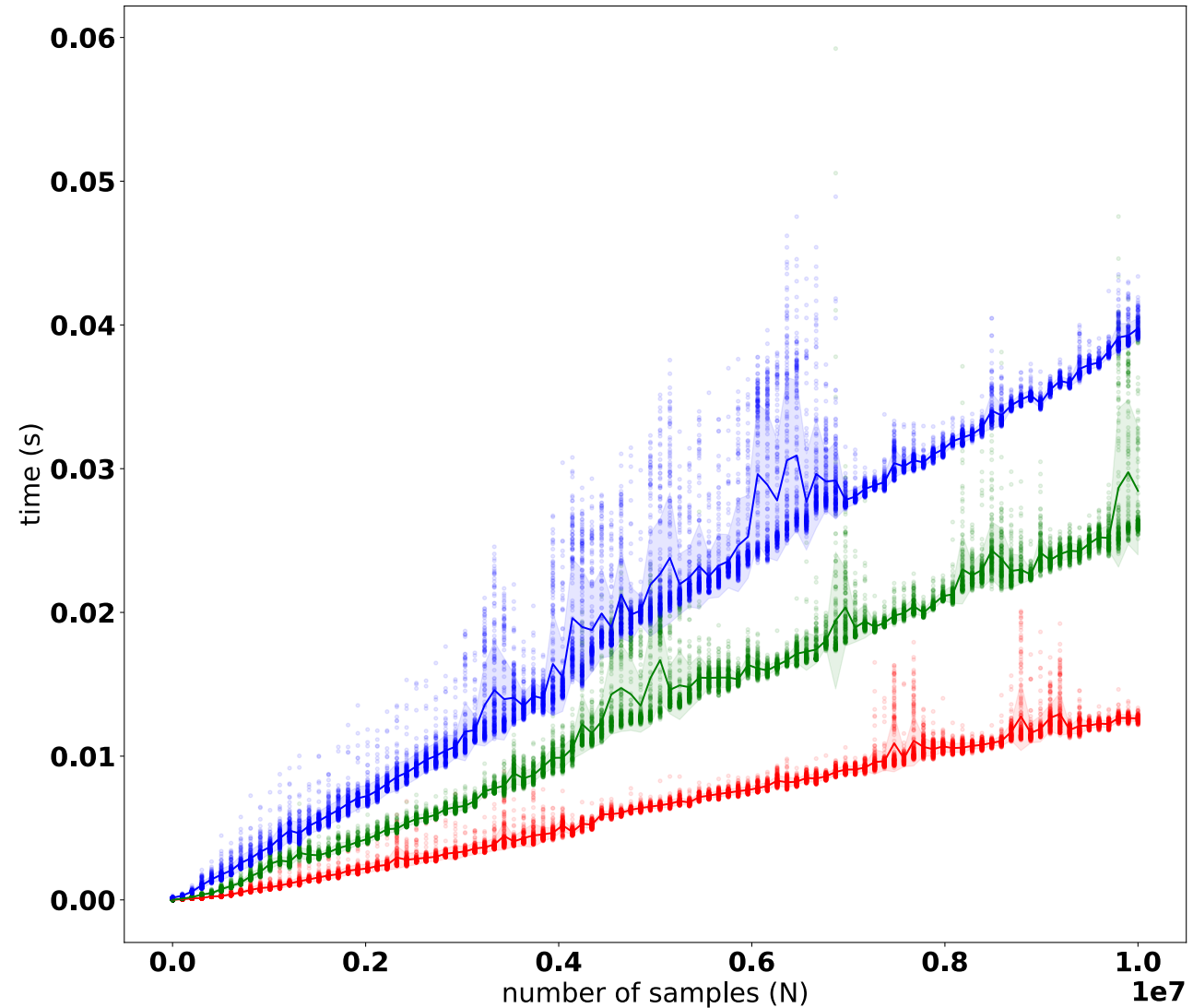
$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

3 floating point operations: $a_1 + b_1$ $a_2 + b_2$ $a_3 + b_3$

- What if $\mathbf{a}, \mathbf{b} \in \mathbb{R}^N$?
- Floating point operations take time, and that time varies across processors
- If we measure the time to calculate $\mathbf{a} + \mathbf{b}$ for $\mathbb{R}^3$, we can approximate the time for $\mathbb{R}^N$

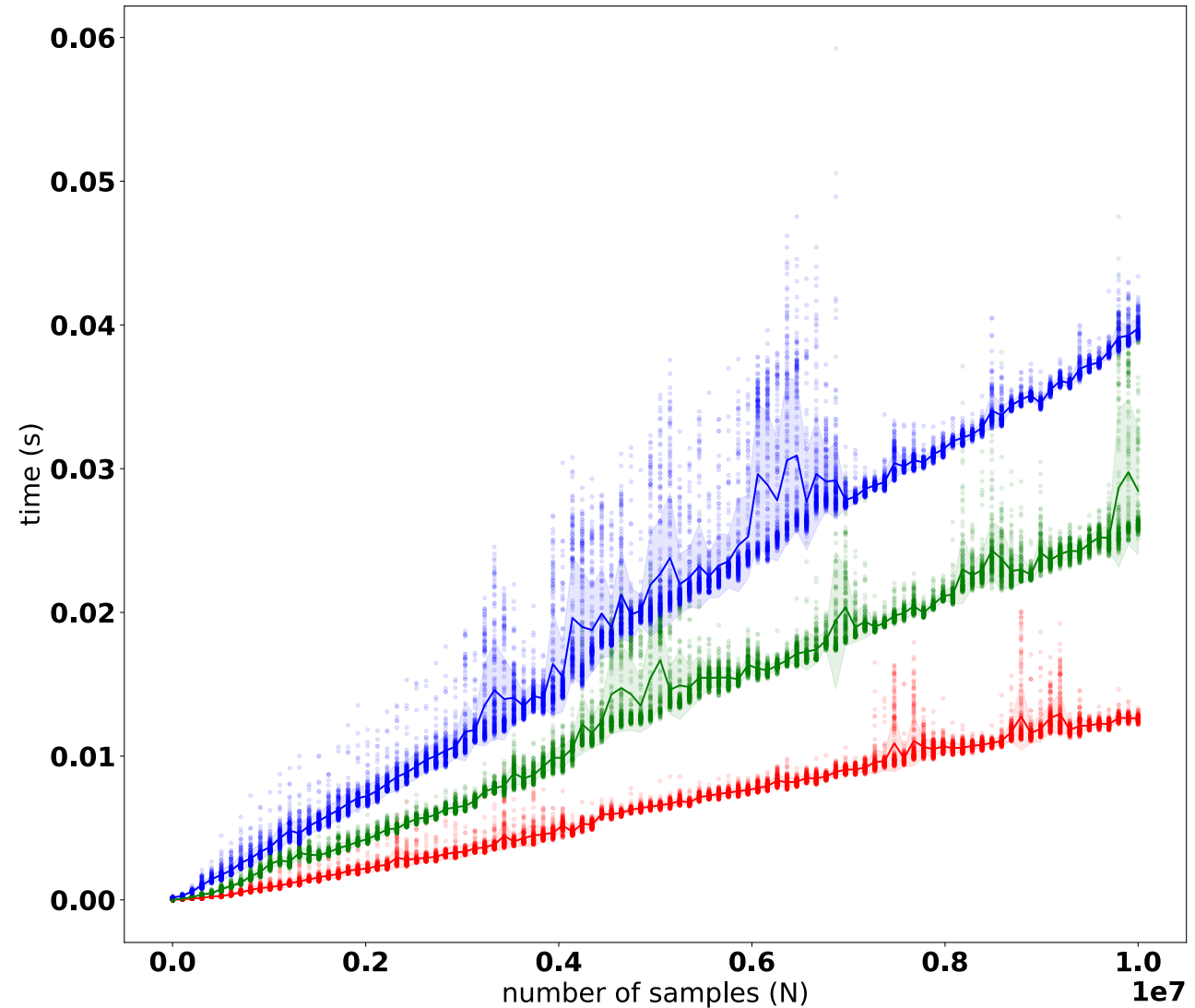
Running an $O(n)$ Algorithm

```
1 float* a = new float[N];  
2 float* b = new float[N];  
3 float* c = new float[N];  
4 float* d = new float[N];  
5 float* e = new float[N];  
6  
7 // perform 1 FLOP vector operation  
8 for(int i = 0; i < N; i++) {  
9     c[i] = a[i] + b[i];  
10 }
```



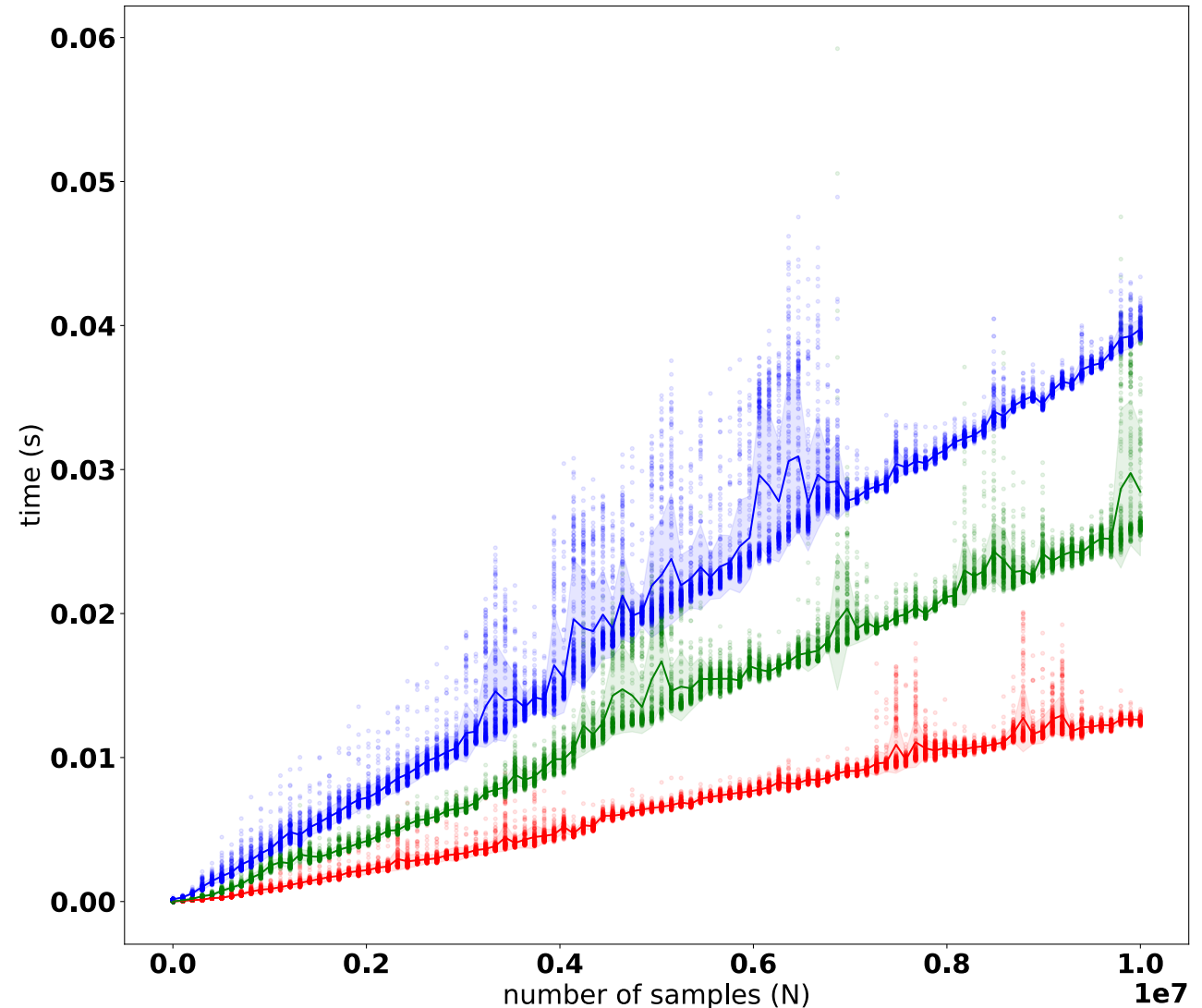
Running an $O(n)$ Algorithm

```
1 float* a = new float[N];
2 float* b = new float[N];
3 float* c = new float[N];
4 float* d = new float[N];
5 float* e = new float[N];
6
7 // perform 1 FLOP vector operation
8 for(int i = 0; i < N; i++) {
9     c[i] = a[i] + b[i];
10 }
11
12 // perform 2 FLOP vector operation
13 for(int i = 0; i < N; i++) {
14     d[i] = (a[i] + b[i]) * c[i];
15 }
```



Running an $O(n)$ Algorithm

```
1 float* a = new float[N];
2 float* b = new float[N];
3 float* c = new float[N];
4 float* d = new float[N];
5 float* e = new float[N];
6
7 // perform 1 FLOP vector operation
8 for(int i = 0; i < N; i++) {
9     c[i] = a[i] + b[i];
10 }
11
12 // perform 2 FLOP vector operation
13 for(int i = 0; i < N; i++) {
14     d[i] = (a[i] + b[i]) * c[i];
15 }
16
17 // perform 2 FLOP vector operation
18 for(int i = 0; i < N; i++) {
19     e[i] = (a[i] + b[i]) * c[i] + d[i];
20 }
```



Running an $O(n^2)$ Algorithm

```
1 float* A = new float[M * N];
2 float* x = new float[N];
3 float* y = new float[N];
4 float* z = new float[N]
5 float alpha, beta;
6
7 // load data
8
9 // perform 1 FLOP matrix multiplication
10 for(int m = 0; m < M; m++) {
11     float(int n = 0; n < N; n++) {
12         y[m] = A[n * M + m] * x[n];
13     }
14 }
```


Running an $O(n^2)$ Algorithm

```
1 float* A = new float[M * N];
2 float* x = new float[N];
3 float* y = new float[N];
4 float* z = new float[N]
5 float alpha, beta;
6
7 // load data
8
9 // perform 1 FLOP matrix multiplication
10 for(int m = 0; m < M; m++) {
11     float(int n = 0; n < N; n++) {
12         y[m] = A[n * M + m] * x[n];
13     }
14 }
15
16 // perform gemv vector operation
17 for(int m = 0; m < M; m++) {
18     float(int n = 0; n < N; n++) {
19         z[m] += alpha * A[n * M + m] * x[n];
20     }
21     y[m] += beta * z[m];
22 }
```

Comparing Algorithmic Complexity

- An $O(n)$ algorithm will beat an $O(n^2)$ algorithm for large n
- Two types of algorithmic optimization:
 1. Reduce computational complexity (ex. $O(n^2) \rightarrow O(n)$). This is a mostly theoretical (mathematical) task.
 2. Improve performance to decrease C . This requires optimization within the source code, hardware, or instructions

Loops and Complexity

$$\mathbf{c} = \mathbf{a} + \mathbf{b}$$

$$\mathbf{c} \in \mathbb{R}^n \quad \mathbf{a} \in \mathbb{R}^n \quad \mathbf{b} \in \mathbb{R}^n$$

```
for(int i = 0; i < N; i++) {  
    c[i] = a[i] + b[i];  
}
```

$$O(N)$$

$$\mathbf{y} = \mathbf{A}\mathbf{x}$$

$$\mathbf{y} \in \mathbb{R}^m \quad \mathbf{A} \in \mathbb{R}^{m \times n} \quad \mathbf{x} \in \mathbb{R}^n$$

```
for(int m = 0; m < M; m++) {  
    for(int n = 0; n < N; n++) {  
        y[m] += A[n * M + m]  
    }  
}
```

$$O(NM)$$

$$O(N^2)$$

if $N = M$

$$\mathbf{C} = \mathbf{A}\mathbf{B}$$

$$\mathbf{A} \in \mathbb{R}^{m \times n} \quad \mathbf{B} \in \mathbb{R}^{m \times k} \quad \mathbf{C} \in \mathbb{R}^{n \times k}$$

```
for(int k = 0; k < K; k++) {  
    for(int m = 0; m < M; m++) {  
        for(int n = 0; n < N; n++) {  
            C[k * N + n] += A[n * M + m]  
                           * B[m * K + k]  
        }  
    }  
}
```

$$O(NMK)$$

$$O(N^3)$$

if $N = M = K$

Complexity of Common Problems

- $O(1)$ - constant time
 - single operation on two values of *constant bit size*
 - accessing a value in memory (RAM), on an SSD, or in a register
 - use of a lookup table
- $O(\log n)$ - logarithmic time
 - binary or alphabetic search
 - traversing a tree (binary, quadtree, octree)
- $O(n)$ - linear time
 - finding an item in an unsorted list
 - multiplying two n -dimensional vectors
- $O(n^2)$ - quadratic time
 - bubble sort algorithm
 - matrix-vector multiplication
- $O(n!)$ - factorial time
 - evaluate all permutations of a set
 - naive Traveling Salesman Problem

Polynomials

Nested Polynomials

Synthetic Division

Horner's Algorithm



Polynomial Evaluation

- Consider an order- n polynomial:

$$p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1} + a_nx^n$$

- How many floating point operations are needed to get $p(x)$?

$$p_2(x) = a_0 + a_1 \cdot x + a_2 \cdot x \cdot x$$

$$2_+ + 3_\times = 5$$

$$p_3(x) = a_0 + a_1 \cdot x + a_2 \cdot x \cdot x + a_3 \cdot x \cdot x \cdot x$$

$$3_+ + 6_\times = 9$$

$$p_4(x) = a_0 + a_1 \cdot x + a_2 \cdot x \cdot x + a_3 \cdot x \cdot x \cdot x + a_4 \cdot x \cdot x \cdot x \cdot x$$

$$4_+ + 10_\times = 14$$

$$p_n(x) \rightarrow n_+ + \left[\frac{n(n+1)}{2} \right]_\times = n + \frac{1}{2}n + \frac{1}{2}n^2 = O(n^2)$$

Nested Polynomial

Reformulate polynomials to a nested form:

$$p_2(x) = a_0 + a_1x + a_2x^2$$

$$p_3(x) = a_0 + a_1x + a_2x^2 + a_3x^3$$

$$p_2(x) = a_0 + (a_1x + x(a_2))$$

$$p_3(x) = a_0 + (a_1x + x(a_2 + x(a_3)))$$

$$p_2(x) = a_0 + (a_1x + x(a_2))$$

$$2_+ + 2_\times = 4$$

$$p_3(x) = a_0 + (a_1x + x(a_2 + x(a_3)))$$

$$3_+ + 3_\times = 4$$

$$p_4(x) = a_0 + (a_1x + x(a_2 + x(a_3 + x(a_4))))$$

$$4_+ + 4_\times = 4$$

$$p_n(x) \rightarrow n_+ + n_\times = 2n = O(n)$$

Polynomial Evaluation Algorithm

```
1 double p(double x, double* a, int n){
2     double p_x = a[0];
3     
4     double xn = a[0];
5     
6     xn *= x;
7     }
8     p_x += xn;
9 }
10 return p_x;
11 }
```

```
1 double p(double x, double* a, int n){
2     double p_x = a[0];
3     
4     p_x = p_x * x + a[i];
5     }
6     return p_x;
7 }
```


Polynomial Division

- Consider the following polynomial division: $\frac{x^4 + 6x^2 - 7x + 8}{x - 2}$

$$x - 2 \overline{) x^4 + 0x^3 + 6x^2 - 7x + 8}$$

$$\underline{-(x^4 - 2x^3)} $$

$$2x^3 + 6x^2$$

$$\underline{-(2x^3 - 4x^2)}$$

$$10x^2 - 7x$$

$$\underline{-(10x^2 - 20x)}$$

$$13x + 8$$

$$\underline{-(13x - 26)}$$

$$34$$

What happens
when $x = 2$?

$$p(x) = x^4 + 6x^2 - 7x + 8$$

$$\begin{aligned} p(2) &= (2)^4 + 6(2)^2 - 7(2) + 8 \\ &= 16 + 24 - 14 + 8 \\ &= 34 \end{aligned}$$

The remainder term is the solution of
the polynomial at the root of the divisor

Nested Polynomials and Division

$$\begin{array}{r}
 x^3 + 2x^2 + 10x + 13 \\
 x - 2 \overline{) x^4 + x^3 + 6x^2 - 7x + 8} \\
 \underline{-(x^4 - x^3)} \\
 3 \\
 + x^2 \\
 \underline{-(2x^3 - x^2)} \\
 x^2 \\
 - x^2 \\
 \underline{-(10x^2 - x)} \\
 x \\
 + x \\
 \underline{-(13x -)} \\
 8
 \end{array}$$

$x^4 + 0x^3 + 6x^2 - 7x + 8$
 $(8 + x(-7 + x(6 + x(0 + x(1)))))$
 $p(2) = (8 + 2(-7 + 2(6 + 2(0 -))))$

$(8 + 2(-7 + 2(6 -)))$
 $(8 + 2(-))$
 (34)

The algorithm for solving $p(c)$ is identical to calculating the remainder of $\frac{p(c)}{x-c}$

Synthetic Division

- Long division for polynomials is often simplified using synthetic division:

$$x - 2 \overline{) x^4 + 0x^3 + 6x^2 - 7x + 8} = x^3 + 2x^2 + 10x + 13 + \frac{34}{x - 1}$$

2	1	0	6	-7	8
	↓	2	4	20	26
	1	2	10	13	34

$$x^3 + 2x^2 + 10x + 13 + \frac{34}{x - 2}$$

Horner's Algorithm

```
1 double horner(double* C, int N, double x){
2     if(N == 0) return 0.0;           // output zero if there are no coefficients
3     px = C[0];                       // initialize the first coefficient
4
5     for(int i = 1; i < N-1; i++){    // for each coefficient
6         px = px * x + C[i];          // evaluate the innermost nested term
7     }
8 }
```

C is an array of length **N** holding the coefficients of $p(x)$

N is the number of coefficients

x is the point for which $p(x)$ is to be evaluated

Calculating Derivatives

$$\frac{x^4 + 6x^2 - 7x + 8}{x - 2} = x^3 + 2x^2 + 10x + 13 + \frac{34}{x - 2}$$

- Consider an n -order polynomial $p_n(x)$: $p_n(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x^1 + a_0$
- Division by $(x - c)$ produces a new polynomial $r_{n-1}(x)$ with a remainder term:

$$\frac{p_n(x)}{x - c} = r_{n-1}(x) + \frac{p(c)}{x - c}$$

- Reformulate and calculate the derivative $p'(x)$:
$$p_n(x) = (x - c)r_{n-1}(x) + p(c)$$
$$p'_n(x) = (x - c)r'_{n-1}(x) + r_{n-1}(c)$$
- We know the term $r_{n-1}(x)$ but do not have $r'_{n-1}(x)$, but setting $x = c$ yields:

$$p'_n(c) = r_{n-1}(c)$$

Horner's Algorithm with Derivatives

- Storing the intermediate terms allows us to calculate the derivative:

```
double horner(double* C, int N, double x, double* R){
    if(N == 0) return 0.0;           // output zero if there are no coefficients
    R[0] = R[0];                     // initialize the first coefficient

    for(int i = 1; i < N-1; i++){    // for each coefficient
        R[i] = R[i-1] * x + C[i];    // evaluate the innermost nested term
    }
}

int N;                             // order of the polynomial
double* P = new double[N + 1];     // array to store coefficients of p(x)
double* R = new double[N + 1];     // array to store coefficients of r(x)
double c;                           // c value for the polynomial
// ...

horner(P, N, c, R);                 // calculate R
double p_c = R[N - 1];              // p(c) (and also remainder term)

horner(R, N, c, R);                 // calculate r(c), overwriting R
double pp_c = R[N-1];               // p'(c)
```